

STREAM PROCESSING AVEC APACHE KAFKA

Projet capstone

Neuf sujets, un seul contrat : encaisser le réel, et le défendre

ESGI · 5^e année · Data Engineering · 2025–2026

PROJET

Objectifs

Construire un pipeline de bout en bout

Ingérer, valider, agréger, enrichir, détecter — sur un vrai cluster partagé, pas sur votre portable.

Encaisser un flux qui n'est jamais propre

7 % de messages cassés, des retards de 30 à 180 min, des doublons. L'application ne doit jamais tomber.

Choisir — et savoir pourquoi

Type de fenêtre, clé d'agrégation, nature de la jointure, sort des retardataires. Chaque décision se défend.

Tenir une soutenance

Modification en direct, exigence métier nouvelle séance tenante. Un ticket que vous n'expliquez pas n'est pas crédité.

01

PARTIE 1

Le cadre

Un seul contrat, neuf terrains

Les sujets changent de domaine, jamais de structure. Ce qui suit vaut pour les neuf.

Deux flux, un référentiel

Deux topics d'événements, sales. Un topic compacté qui sert de table de référence — et qui porte une information qu'aucun événement ne contient.

Une DLQ motivée

Tout message invalide part en dead letter queue, avec le message original et la raison du rejet. Sans exception, sans crash.

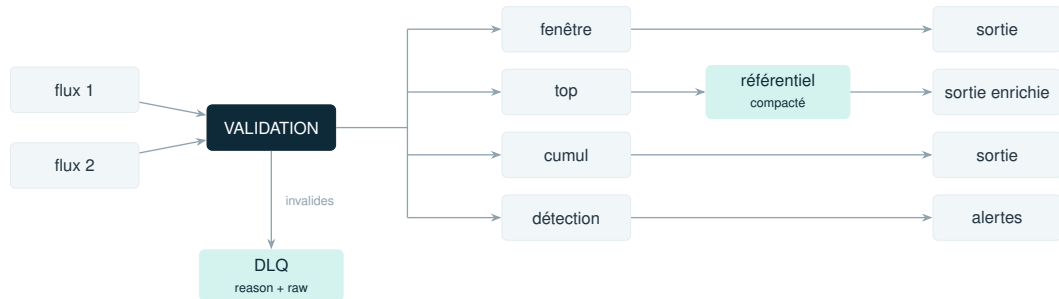
Cinq sorties, préfixées

`<grp>.<projet>.<sortie>`. Le préfixe groupe et l'`application.id` sont déjà câblés dans `Topics.java`.

Six tickets

Un socle obligatoire à 8 points, cinq bonus à 12 points. Plafond 20. Les bonus ne comptent que défendus.

La forme imposée



La validation d'abord

Un message rejeté en DLQ ne doit alimenter aucune sortie. Un montant invalide ne se facture pas.

La jointure avant l'agrégat

Le référentiel porte la clé de regroupement. Joindre après le `groupBy`, c'est trop tard.

Le flux ne sera jamais propre

Anomalie	Exemple	Ce qu'elle casse si vous l'ignorez
Champ requis absent ou null	pas de <code>campaign_id</code>	<code>NullPointerException</code> , donnée non attribuable
Mauvais type	<code>"rpm": "douze"</code>	crash de désérialisation
Le type piégeux	<code>"rpm": true</code>	en JSON, <code>true</code> n'est pas un nombre
Valeur hors bornes	<code>amount_eur: 1e9</code>	un chiffre d'affaires absurde
JSON tronqué, vide, non-JSON	<code>{"unit_id": "u</code>	poison pill : l'appli meurt en boucle
Événement en retard	horodatage -30 à 180 min	fenêtres faussées — et pire, selon le sujet
Doublon exact	même identifiant deux fois	le client est facturé deux fois
Clé hors référentiel	<code>smurf-0007</code>	données perdues si <code>join</code> au lieu de <code>leftJoin</code>

Environ 7 % des messages, plus les retards et les doublons. Ce n'est pas un bug du générateur : c'est le sujet.

D'où vient le flux

Le cluster partagé

Le générateur y tourne en continu, rétention 24 h. C'est là que se fait l'évaluation. Vos topics de sortie y sont préfixés par votre groupe.

Le jeu de données figé

Un `.jsonl` par topic, distribué en début de projet. De quoi démarrer sans cluster, et travailler tous sur la même donnée.

Rejouer le jeu de données dans votre cluster local

```
docker compose up -d # 3 brokers KRaft + Kafbat UI sur :8080

python rejouer.py --dossier data/<projet> --project <projet> \
  --create-topics --replication-factor 1 --bootstrap localhost:29092
```

Le rejeu repose l'horodatage d'origine sur chaque message : les retardataires restent en retard.

Huit points, et rien d'autre à négocier

Le ticket 1 est le même partout : consommer les deux flux, valider, router les invalides. Il vaut **8 points sur 20**. Sans lui, le reste ne compte pas.

10 minutes sans crash

En production, sur le cluster partagé. Pas une démo locale de trente secondes.

Une DLQ motivée

Message original + raison lisible.
On compte les raisons distinctes.

Aucun valide perdu

Ce qui entre valide doit ressortir. Une validation trop zélée coûte aussi cher.

Un projet qui rend 8/20 propres vaut mieux qu'un projet à cinq tickets qui tombe au bout de deux minutes.

02

PARTIE 2

Les sujets

Les neuf sujets

Sujet	Modèle	Stack	Le morceau
BINGE	Netflix	Java	sessions de visionnage, API Interactive Queries
CARTFLOW	Amazon	Spring Boot	jointure commandes × paiements, stock, exactly-once
DISPATCH	Uber	Quarkus	offre/demande par zone, KTable à soustracteur
VIRAL	Instagram	Java	<code>flatMap</code> hashtags, deux repartitions successives
TEMPO	Spotify	Quarkus	taux de skip, royalties, ferme à streams
SENTINEL	Stripe	Spring Boot	voyage impossible, moyenne mobile, exactly-once
RELAY	Criteo	Quarkus	tunnel impression → clic, jointure flux-flux
ARENA	Riot Games	Quarkus	fenêtres de session sans identifiant, <code>suppress</code>
FORGE	Siemens	Quarkus	détecter le silence d'une machine

Un sujet par groupe. Le framework n'est pas négociable : il fait partie du sujet.

Le backlog type

Ticket	Ce qu'il demande	Ce qu'il évalue	Points
1	Ingestion, validation, DLQ	robustesse, poison pill	8
2	Un ratio sur fenêtre tumbling, avec seuil de significativité	agrégat à deux compteurs, et savoir se taire	+2
3	Un top par catégorie, hopping 15/5, enrichi par GlobalKTable	jointure, <code>leftJoin</code> , placement	+3
4	Un cumul métier par entité — c'est ce qui part en facturation	KTable, dédoublonnage, règle métier	+3
5	Une détection, propre à votre sujet	le morceau technique	+2
6	Le bonus du sujet : tests, exactly-once, API, <code>suppress</code>	maîtrise de l'outil	+2

Les tickets se font **dans l'ordre**. Le ticket 4 est le cœur métier de tous les sujets : c'est celui qui coûte de l'argent quand il est faux.

03

PARTIE 3

L'évaluation

Comment se calcule votre note

Socle : 8 points

Ticket 1 en production, 10 minutes sans crash, DLQ motivée. C'est acquis ou ça ne l'est pas.

Un ticket non défendu = 0

Même s'il tourne. Même s'il est juste. Même si le correcteur l'a validé. Ce n'est pas une menace, c'est le barème.

Bonus : +12 au maximum

Tickets 2 à 6. Chacun est vérifié automatiquement sur vos topics de sortie, puis défendu.

Le plafond est à 20

Inutile de sur-livrer. Cinq tickets bien compris valent mieux que six recopiés.

Sans aucun bonus défendu, votre maximum est **8/20**.

Comment ça se passe

01

Avant le code

Deux ou trois questions, écran fermé. Sur votre sujet, pas sur Kafka en général.

02

La démo

Votre pipeline tourne. On regarde les topics de sortie, pas les slides.

03

Le code

Ticket par ticket. Pourquoi cette fenêtre, cette clé, cette jointure.

04

Le twist

Une exigence métier nouvelle, à implémenter devant nous. Le Product Owner a changé d'avis.

Tous les membres du groupe répondent

N'importe qui peut être interrogé sur n'importe quel ticket. Un groupe où une seule personne sait répondre est un groupe où une seule personne a travaillé — et la note le dira.